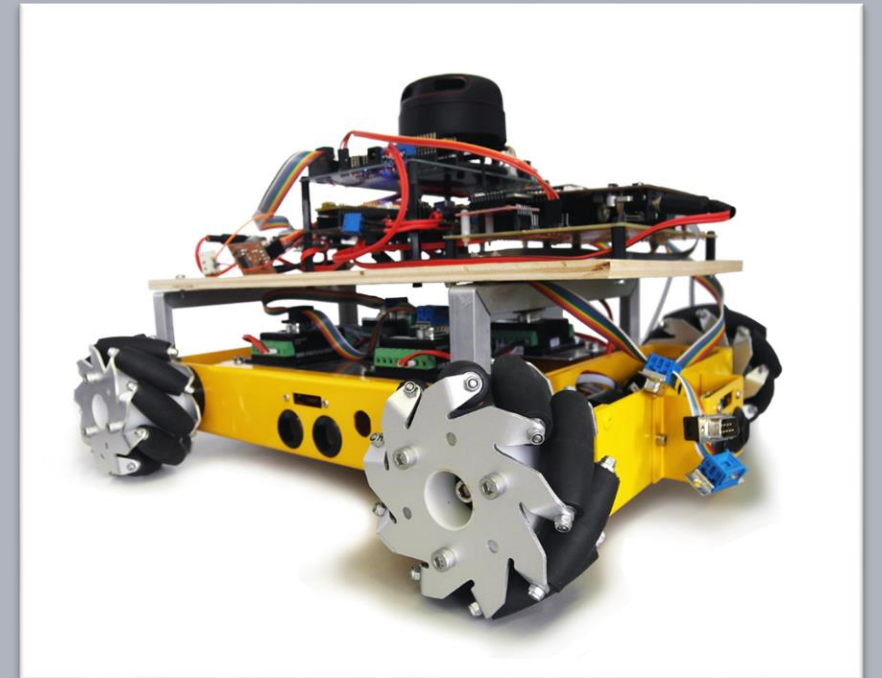


Advanced Embedded Architectures and Applications

Qualification, Test and Debug

Prof. Dr.-Ing. Peter Fromm



Content

- Bug categories
- Debugging the ultrasonic signal
- Another Aurix Bugstory



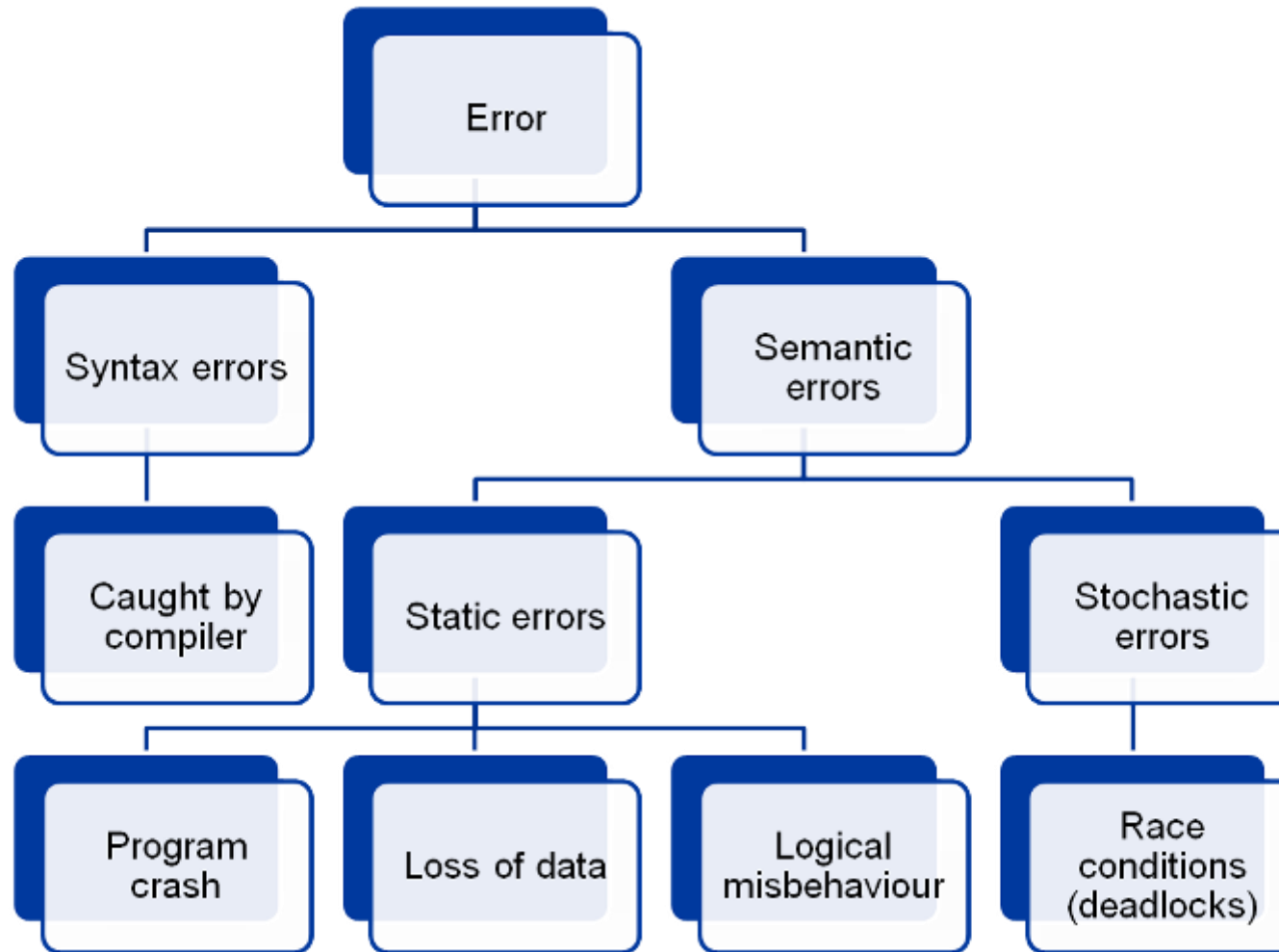
Observation

- The car runs fine for about 15min
- After that time, suddenly the AntiCollision Feature stops to work

A **software bug** is an error, flaw, mistake, failure, fault or “undocumented feature” in a computer program that prevents it from behaving as intended (e.g. , producing an incorrect or unexpected result).

Most bugs arise from mistakes and errors made by people in either a program's source code or its design, and a few are caused by compilers producing incorrect code. A program that contains a large number of bugs, and/or bugs that seriously interfere with its functionality, is said to be buggy. Reports detailing bugs in a program are commonly known as bug reports, fault reports, problem reports, trouble reports, change requests, and so forth.

Error classification



Questions

- How can we get the bug reproducible?
- How can we determine the buggy section?
- What is the root cause?

- Do we have a single ore more bugs???
- Can we introduce some kind of workaround to continue working?



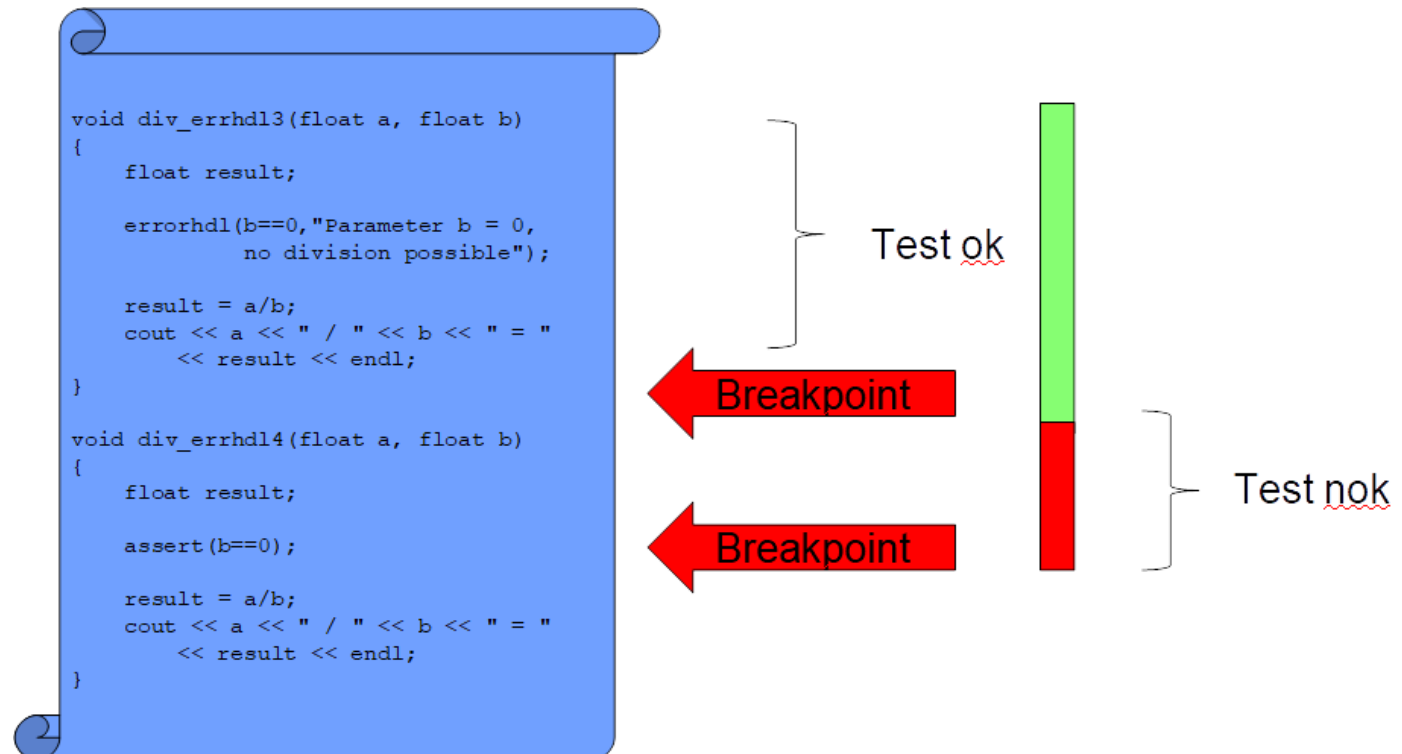
Reproducibility

- Reproducibility means, that we know the conditions leading to the bug. I.e. can bring the system into the buggy state.
- This is especially important for stochastically appearing bugs, as otherwise we have no means of qualifying our bug fix.
- A good architecture and tracing capabilities will help.

Believing != Knowing

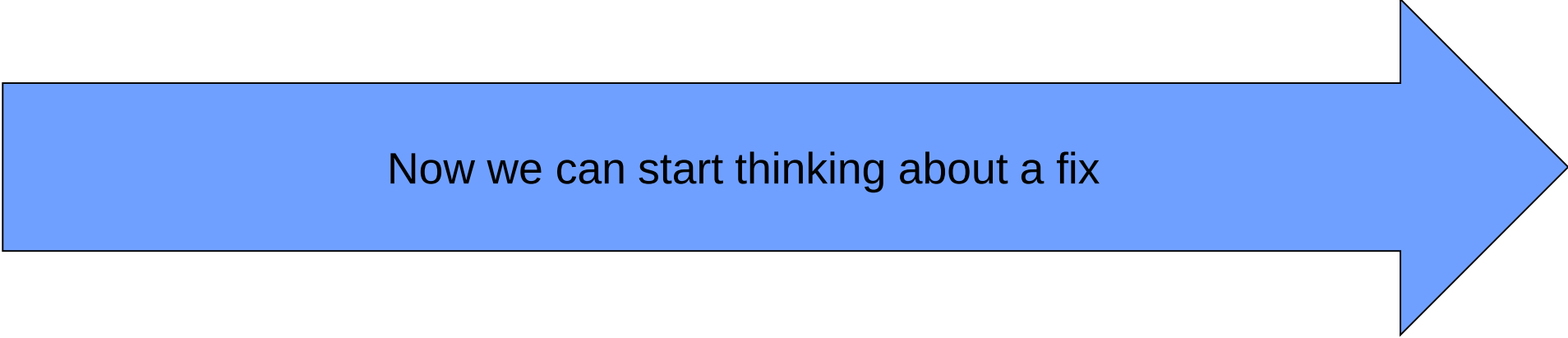
Buggy section

- In order to make debugging efficient, we must identify the (sub-) part of the program which might cause the bug.
- This can be done by analyzing dependencies in the architecture (review) or by using techniques like slicing



Root Cause

An error root cause is the exact knowledge of the **error conditions** and the **code of the program generating the error** and the **knowledge why this code generated** the error (e. g. because a division by zero is not supported by the compiler/controller)



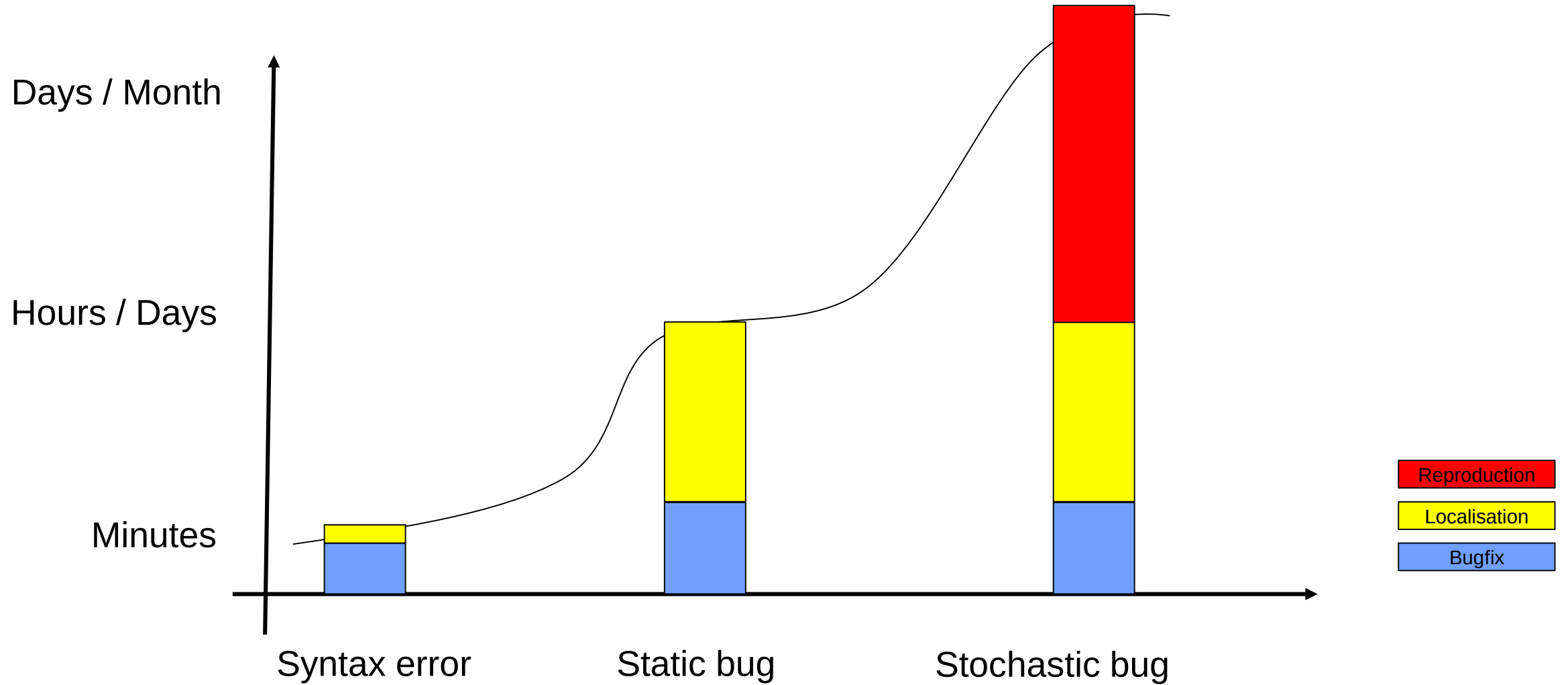
Now we can start thinking about a fix

Workaround

- Sometimes, we may not have sufficient resources for a proper bugfix.
- As a first fix, a workaround may be introduced, e.g.
 - Deactivation of the buggy feature
 - Adding of data clippers
 - Adding of monitors

A workaround is not a proper fix. It only gives you time to find the real bug. This still needs to be done!

Time flies by....



Typical causes for runtime bugs

Data

- Processing of wrong data (e.g. division by zero)
- Working with uninitialized data
- Wrong type casting resulting in a loss of data
- Off-by-One (for (i=0; i<= a; i++))
- Assignment versus equality check (if a=b)

Pointers

- Working with the address instead of the content
- Working with deallocated memory

Illegal configuration

- Typical problem in a component based development
- Wrong configuration of a reused component
- E.g. configuration says 3 CAN controllers, but microcontroller only has 2

Static

Typical causes for runtime bugs

State

- Unexpected state change

Multitasking

- Unprotected access to shared data or other critical regions
- Deadlocks

Realtime

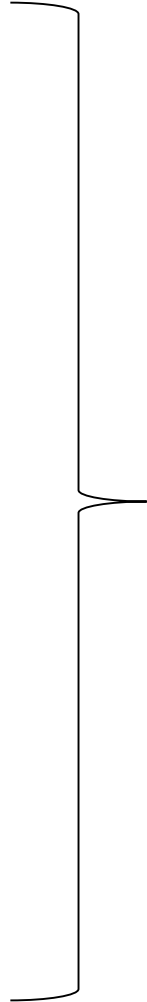
- Violated realtime constraints

Buffers

- Over- or underflow
- Data corruption

Hardware

- Missing error handlers
- Wrong synchronisation



Stochastic

Some general strategies before we start debugging

- Create a copy of the initial buggy version of the program.
 - Don't touch this code from now on! Only work in the copy!
 - During debugging, you will change the code
 - Undoubtedly, you will also introduce wrong changes
 - And you want to test your system later on against the original system
- Be open minded
 - You have no clue where the problem is located
 - Accept this!
 - Probably you are bug-blind concerning your own code!
 - Bring in a second pair of eyes early
 - (before you jump out the window)



Let's create some data to analyze – a long run of the program

Observation 1

HTerm 0.8.1beta

File Options View Help

Connect Port COM12 R Baud 115200 Data 8 Stop 1 Parity None ☐ CTS Flow control

Rx 3163238 Reset Tx 0 Reset Count 0 69 Reset Newline at LF ☒ Show newline characters

☐ Clear received ☒ Ascii ☐ Hex ☐ Dec ☐ Bin Save output ☐ Clear at 0 Newline every ... characters ☒ Autoscroll ☐ Show errors Newline after ... ms receive pause (0=off) 0 CTS DSR RI DCD

Sequence Overview x Received Data

1	5	10	15	20	25	30	35	40	45	50	55	60	65	70	75	80	85	90	95	100	105	110	115	120	125	130	
CONTROL_limitSpeed_run : Front: 73 Rear: 206_v																											
CONTROL__Drive : 0_v																											
CONTROL_limitSpeed_run : Front: 73 Rear: 205_v																											
CONTROL_limitSpeed_run : Front: 73 Rear: 205_v																											
CONTROL__Drive : 0_v																											
CONTROL_limitSpeed_run : Front: 73 Rear: 205_v																											
CONTROL_limitSpeed_run : Front: 73 Rear: 205_v																											
CONTROL__Drive : 0_v																											
CONTROL_limitSpeed_run : Front: 73 Rear: 205_v																											
CONTROL_limitSpeed_run : Front: 73 Rear: 205_v																											
CONTROL__Drive : 0_v																											
CONTROL_limitSpeed_run : Front: 205 Rear: 205_v																											
CONTROL_limitSpeed_run : Front: 205 Rear: 205_v																											
CONTROL__Drive : 0_v																											
CONTROL_limitSpeed_run : Front: 205 Rear: 73_v																											
CONTROL_limitSpeed_run : Front: 205 Rear: 73_v																											
CONTROL__Drive : 0_v																											
CONTROL_limitSpeed_run : Front: 499 Rear: 73_v																											
CONTROL_limitSpeed_run : Front: 499 Rear: 73_v																											
CONTROL__Drive : 0_v																											
CONTROL_limitSpeed_run : Front: 499 Rear: 208_v																											
CONTROL_limitSpeed_run : Front: 499 Rear: 208_v																											

Selection (-)

Input control

Input options

☒ Ascii ☐ Hex ☐ Dec ☐ Bin Send on enter None

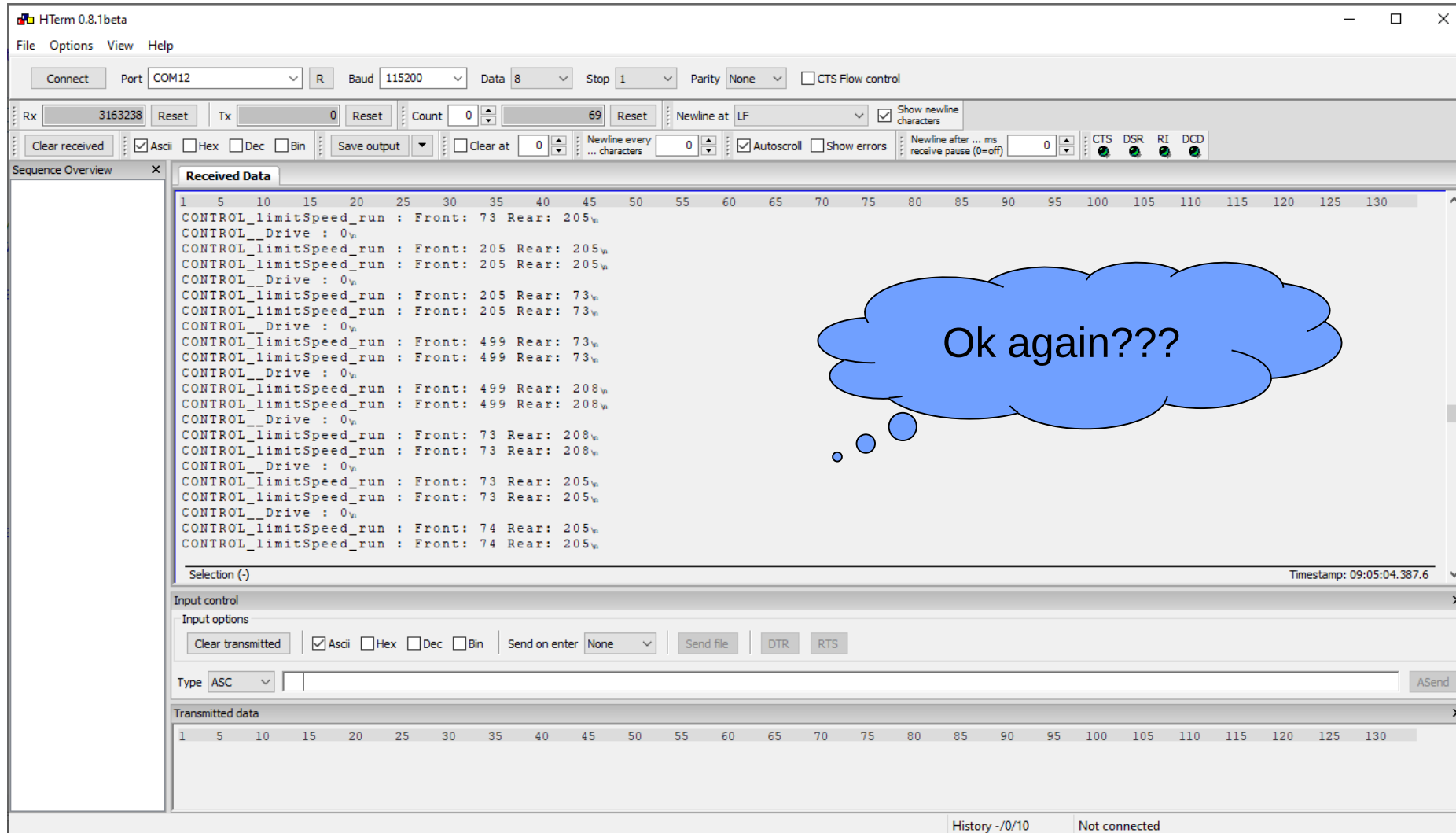
Type ASC

Transmitted data

1	5	10	15	20	25	30	35	40	45	50	55	60	65	70	75	80	85	90	95	100	105	110	115	120	125	130
---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	-----	-----	-----	-----	-----	-----

History -/0/10 Not connected

Observation 2



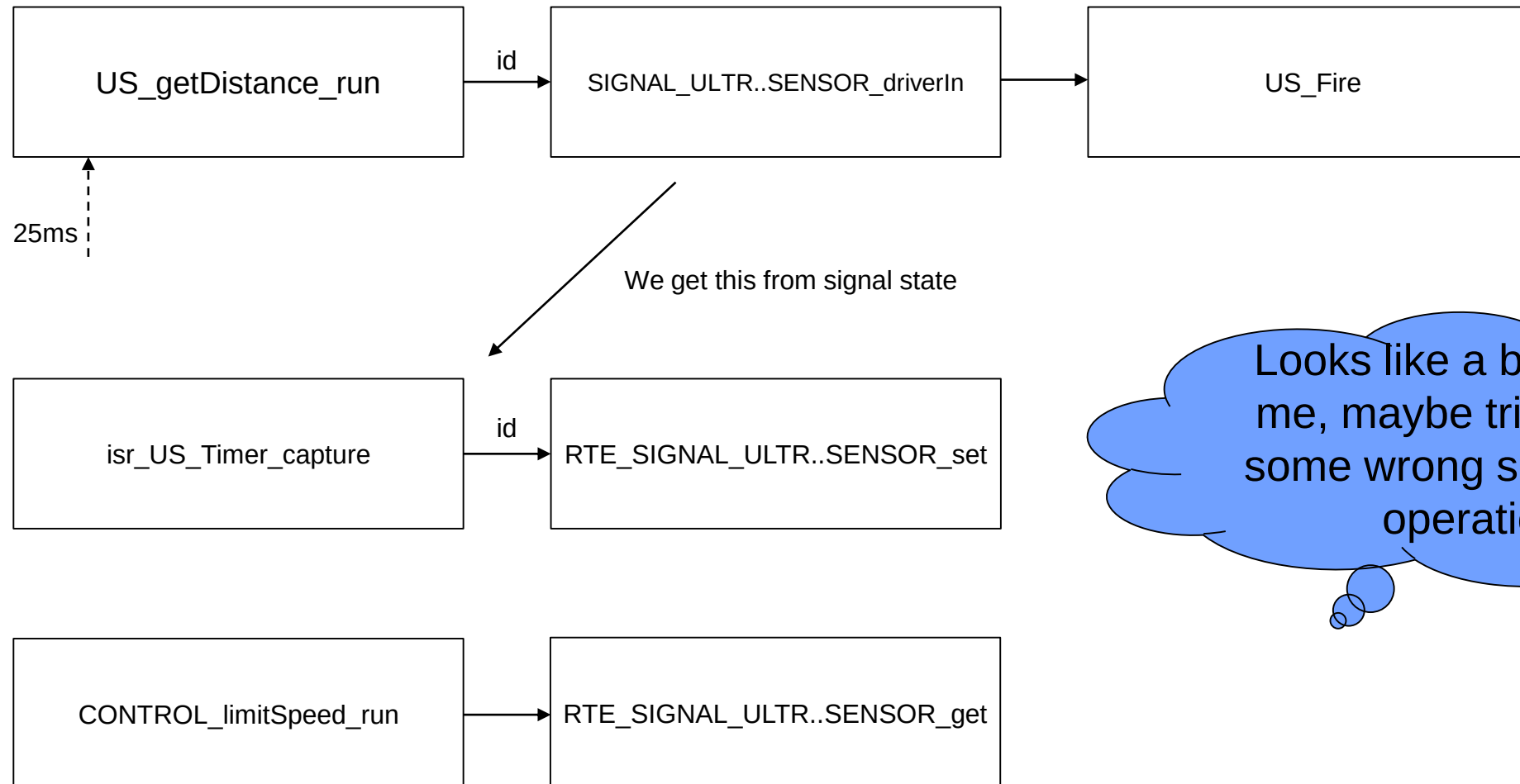
Observation 3

The screenshot shows the HTerm 0.8.1beta interface. The 'Received Data' pane displays a sequence of control messages. A blue thought bubble with the text 'Front and rear values are flipped' points to the 'Front' and 'Rear' fields in the messages. The messages are as follows:

Line	Message
1	CONTROL_limitSpeed_run : Front: 49 Rear: 74 _v
2	CONTROL_limitSpeed_run : Front: 49 Rear: 74 _v
3	CONTROL__Drive : 0 _v
4	CONTROL_limitSpeed_run : Front: 49 Rear: 74 _v
5	CONTROL_limitSpeed_run : Front: 49 Rear: 74 _v
6	CONTROL__Drive : 0 _v
7	CONTROL_limitSpeed_run : Front: 43 Rear: 74 _v
8	CONTROL_limitSpeed_run : Front: 43 Rear: 74 _v
9	CONTROL__Drive : 0 _v
10	CONTROL_limitSpeed_run : Front: 43 Rear: 74 _v
11	CONTROL_limitSpeed_run : Front: 43 Rear: 74 _v
12	CONTROL__Drive : 0 _v
13	CONTROL_limitSpeed_run : Front: 205 Rear: 74 _v
14	CONTROL_limitSpeed_run : Front: 205 Rear: 74 _v
15	CONTROL__Drive : 0 _v
16	CONTROL_limitSpeed_run : Front: 205 Rear: 74 _v
17	CONTROL_limitSpeed_run : Front: 205 Rear: 74 _v
18	CONTROL__Drive : 0 _v
19	CONTROL_limitSpeed_run : Front: 205 Rear: 74 _v
20	CONTROL_limitSpeed_run : Front: 205 Rear: 74 _v
21	CONTROL__Drive : 0 _v
22	CONTROL_limitSpeed_run : Front: 205 Rear: 74 _v

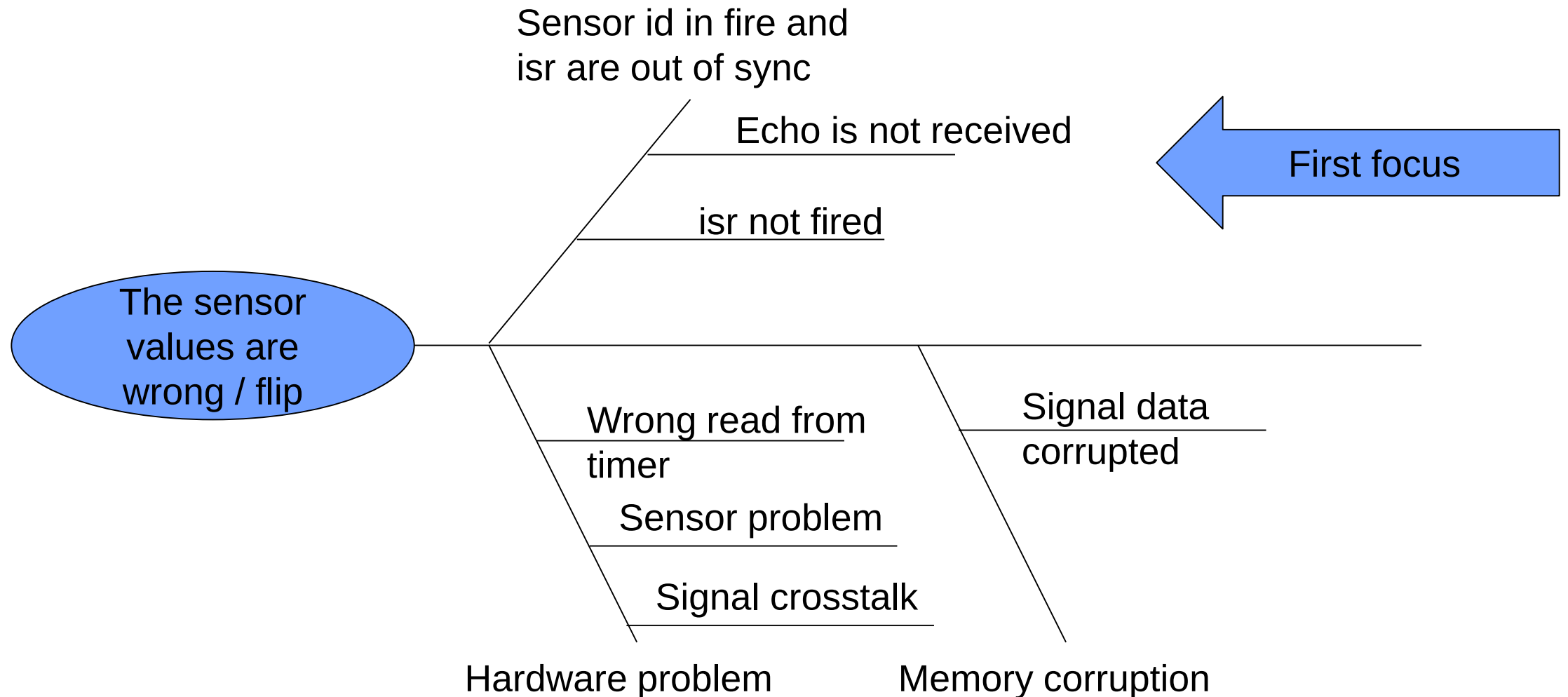
The 'Input control' pane shows 'Input options' with 'Clear transmitted', 'Ascii', 'Hex', 'Dec', 'Bin', 'Send on enter' (None), 'Send file', 'DTR', and 'RTS'. The 'Type' is set to 'ASC'. The 'Transmitted data' pane is empty. The status bar at the bottom shows 'History -/0/10' and 'Not connected'.

Let's do some systematic brainstorming



Looks like a bad race to me, maybe triggered by some wrong sensor read operation?

Ishikawa Diagram



Code review

```
US_ReadDistance_isr(&distance);

//Now we have to write the distance to the correct signal
//In the pullport function, the corresponding signal status has been set to US_ISMEASURING
//The User has to make sure, that only one signal is being fired at the same time
//Additional locks may be added to the pullport, i.e. return an error if one of the other signals is in the IS_MEASURING state.
//But this seems a bit hacky...
//Todo: Find a better solution

//LOG_I("Isr","D = %d",distance);

SIGNAL_ULTRASONICSENSOR_data_t us = SIGNAL_ULTRASONICSENSOR_INIT_DATA;

//Check for front sensor
us = RTE_SIGNAL_ULTRASONICSENSOR_get(&SO_USFRONT_signal);
if (us.m_status == US_ISMEASURING)
{
    us.m_distance = distance;
    us.m_status = US_ISVALID;
    RTE_SIGNAL_ULTRASONICSENSOR_set(&SO_USFRONT_signal, us);
}

//Check for rear sensor
us = RTE_SIGNAL_ULTRASONICSENSOR_get(&SO_USREAR_signal);
if (us.m_status == US_ISMEASURING)
{
    us.m_distance = distance;
    us.m_status = US_ISVALID;
    RTE_SIGNAL_ULTRASONICSENSOR_set(&SO_USREAR_signal, us);
}
```

I would expect a resync?

Adding some debug code....

As expected. Some echos were not received, but this does not cause a problem.

```
//Check for invalid states
if ( (US_ISVALID == us_front.m_status && US_ISVALID == us_rear.m_status) ||
    (US_ISMEASURING == us_front.m_status && US_ISMEASURING == us_rear.m_status))
{
    LOG_E("isr_US_Timer_capture", "wrong signal states front (%d), rear (%d)", us_front.m_status, us_rear.m_status);

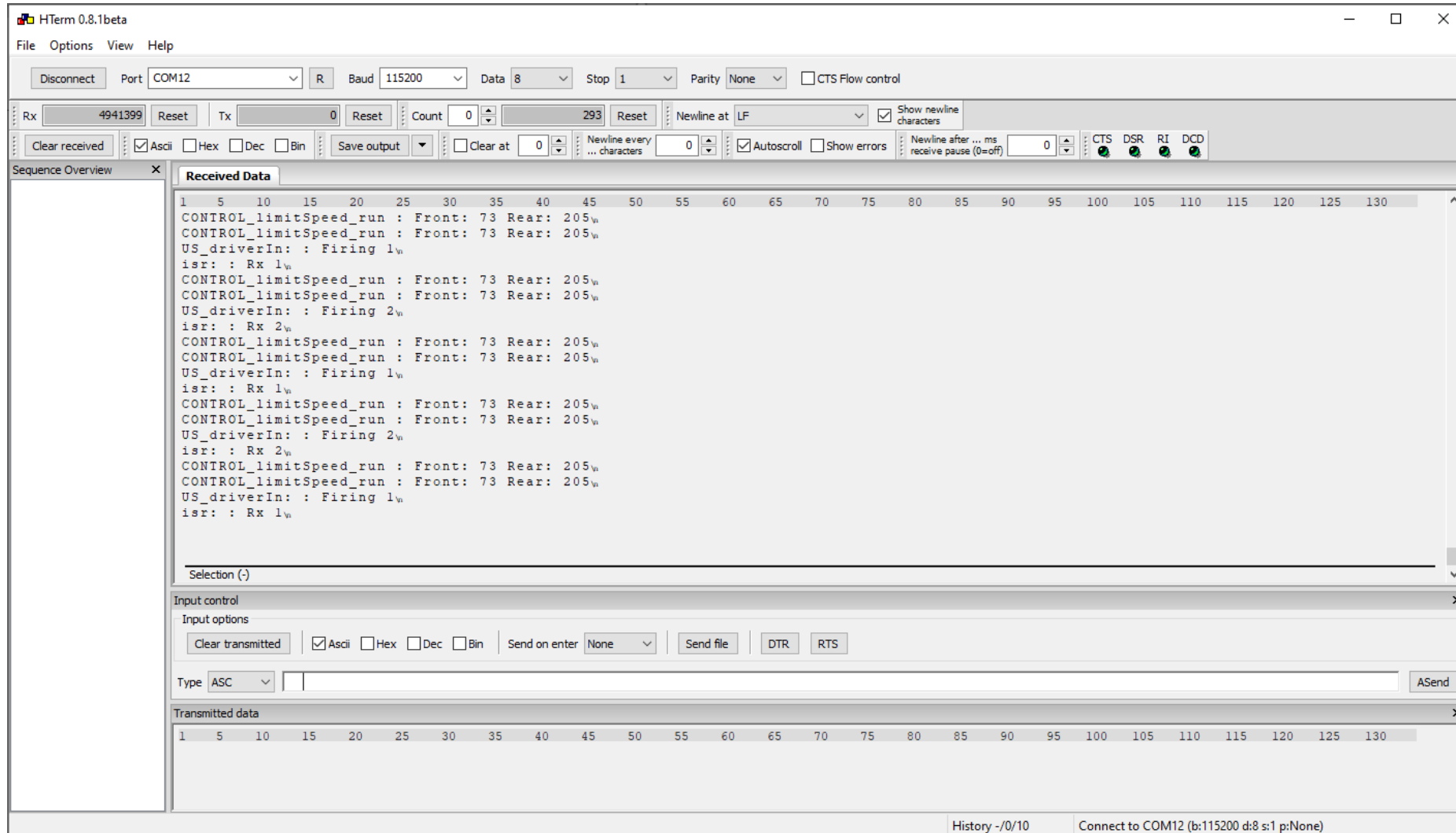
    us_rear.m_status = US_ISINVALID;
    us_front.m_status = US_ISINVALID;

    RTE_SIGNAL_ULTRASONICSENSOR_set(&SO_USFRONT_signal, us_front);
    RTE_SIGNAL_ULTRASONICSENSOR_set(&SO_USREAR_signal, us_rear);
}
else
{
    //Check for front sensor
    if (us_front.m_status == US_ISMEASURING)
    {
        us_front.m_distance = distance;
        us_front.m_status = US_ISINVALID;
        RTE_SIGNAL_ULTRASONICSENSOR_set(&SO_USFRONT_signal, us_front);
    }

    //Check for rear sensor
    if (us_rear.m_status == US_ISMEASURING)
    {
        us_rear.m_distance = distance;
        us_rear.m_status = US_ISINVALID;
        RTE_SIGNAL_ULTRASONICSENSOR_set(&SO_USREAR_signal, us_rear);
    }
}
```

1	5	10	15	20	25	30	35	40	45	50	55	60	65
CONTROL_limitSpeed_run : Front: 73 Rear: 205 _{µs}													
CONTROL_limitSpeed_run : Front: 73 Rear: 205 _{µs}													
CONTROL_limitSpeed_run : Front: 73 Rear: 205 _{µs}													
CONTROL_limitSpeed_run : Front: 73 Rear: 205 _{µs}													
CONTROL_limitSpeed_run : Front: 73 Rear: 205 _{µs}													
CONTROL_limitSpeed_run : Front: 73 Rear: 205 _{µs}													
CONTROL_limitSpeed_run : Front: 73 Rear: 205 _{µs}													
isr_US_Timer_capture : wrong signal states for front (0), rear (0) _{µs}													
CONTROL_limitSpeed_run : Front: 73 Rear: 205 _{µs}													
CONTROL_limitSpeed_run : Front: 73 Rear: 205 _{µs}													
CONTROL_limitSpeed_run : Front: 76 Rear: 205 _{µs}													
CONTROL_limitSpeed_run : Front: 76 Rear: 205 _{µs}													
CONTROL_limitSpeed_run : Front: 76 Rear: 205 _{µs}													
CONTROL_limitSpeed_run : Front: 76 Rear: 205 _{µs}													
CONTROL_limitSpeed_run : Front: 76 Rear: 205 _{µs}													
CONTROL_limitSpeed_run : Front: 73 Rear: 205 _{µs}													
CONTROL_limitSpeed_run : Front: 73 Rear: 205 _{µs}													
CONTROL_limitSpeed_run : Front: 73 Rear: 205 _{µs}													
CONTROL_limitSpeed_run : Front: 73 Rear: 205 _{µs}													
CONTROL_limitSpeed_run : Front: 73 Rear: 205 _{µs}													
CONTROL_limitSpeed_run : Front: 73 Rear: 205 _{µs}													
CONTROL_limitSpeed_run : Front: 73 Rear: 205 _{µs}													
CONTROL_limitSpeed_run : Front: 73 Rear: 205 _{µs}													
CONTROL_limitSpeed_run : Front: 73 Rear: 205 _{µs}													

Heisenbugs – Adding some more LOGs, and the problem disappeared



Luckily only for an hour...

Note the
amount of
logging data!

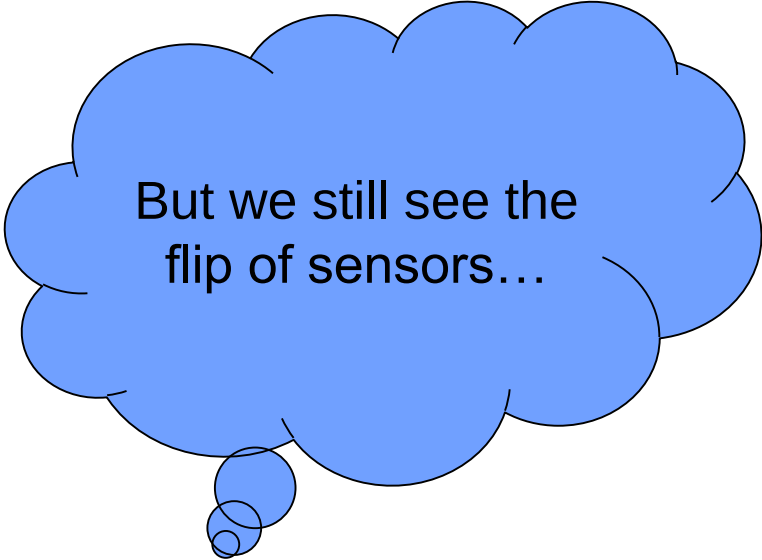
```
US_driverIn: : Firing 2_v
isr: : Rx 2_v
CONTROL_limitSpeed_run : Front: 77 Rear: 153_v
CONTROL_limitSpeed_run : Front: 77 Rear: 153_v
US_driverIn: : Firing 1_v
CONTROL_limitSpeed_run : Front: 77 Rear: 153_v
CONTROL_limitSpeed_run : Front: 77 Rear: 153_v
US_driverIn: : Firing 2_v
isr_US_Timer capture : wrong signal states front (0), rear (0)_v
CONTROL_limitSpeed_run : Front: 77 Rear: 153_v
CONTROL_limitSpeed_run : Front: 77 Rear: 153_v
US_driverIn: : Firing 1_v
isr: : Rx 1_v
CONTROL_limitSpeed_run : Front: 583 Rear: 153_v
CONTROL_limitSpeed_run : Front: 583 Rear: 153_v
US_driverIn: : Firing 2_v
isr: : Rx 2_v
CONTROL_limitSpeed_run : Front: 583 Rear: 73_v
CONTROL_limitSpeed_run : Front: 583 Rear: 73_v
US_driverIn: : Firing 1_v
isr: : Rx 1_v
CONTROL_limitSpeed_run : Front: 153 Rear: 73_v
```

We are firing both sensors

Looks strange – invalid range
Max value is 300 (approx.)

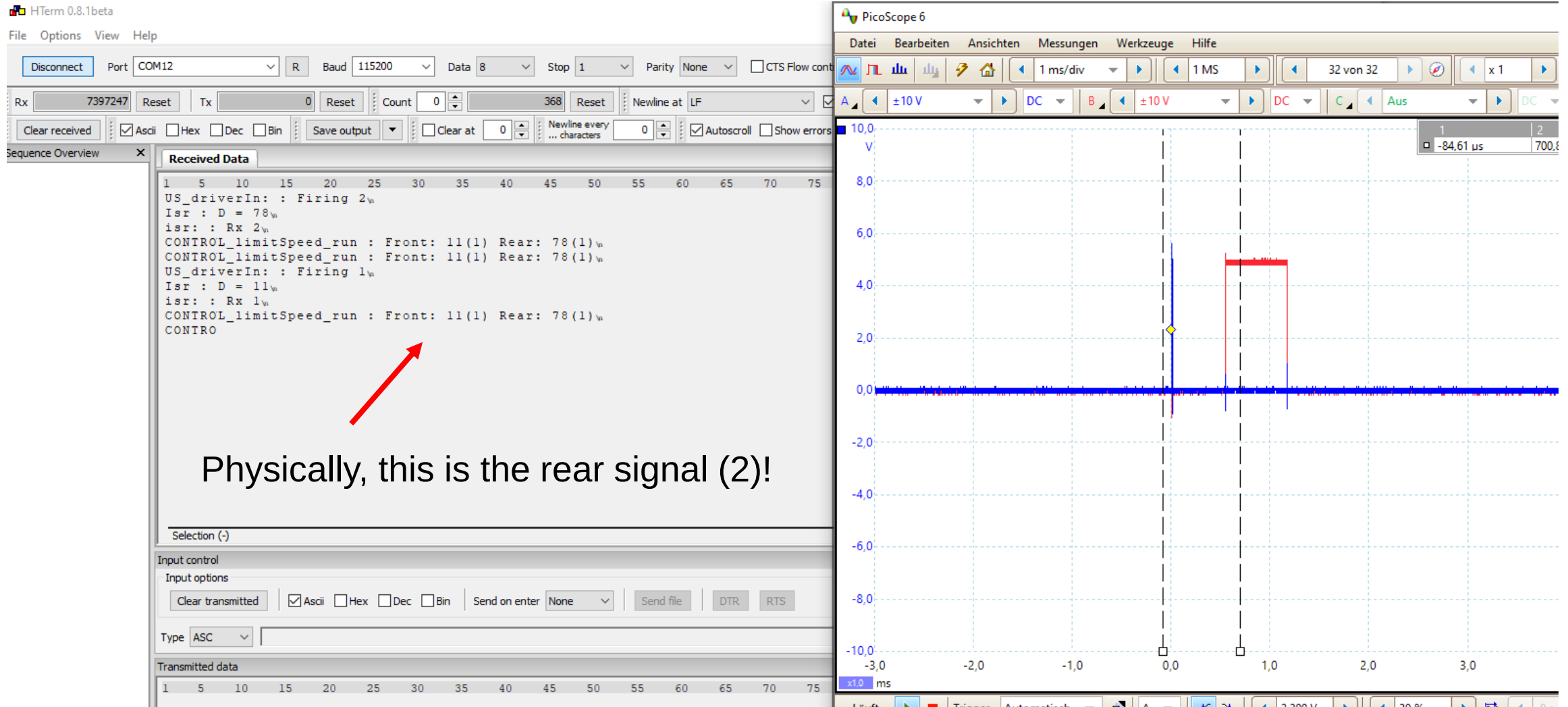
Stranger – the software part looks ok, but the measured data not really

```
CONTROL_limitSpeed_run : Front: 73(1) Rear: 205(1)<\n>
CONTROL_limitSpeed_run : Front: 73(1) Rear: 205(1)<\n>
US_driverIn: : Firing 2<\n>
CONTROL_limitSpeed_run : Front: 73(1) Rear: 205(0)<\n>
CONTROL_limitSpeed_run : Front: 73(1) Rear: 205(0)<\n>
Isr : D = 205<\n>
isr: : Rx 2<\n>
US_driverIn: : Firing 1<\n>
Isr : D = 73<\n>
isr: : Rx 1<\n>
CONTROL_limitSpeed_run : Front: 73(1) Rear: 205(1)<\n>
CONTROL_limitSpeed_run : Front: 73(1) Rear: 205(1)<\n>
US_driverIn: : Firing 2<\n>
Isr : D = 43<\n>
isr: : Rx 2<\n>
CONTROL_limitSpeed_run : Front: 73(1) Rear: 43(1)<\n>
CONTROL_limitSpeed_run : Front: 73(1) Rear: 43(1)<\n>
US_driverIn: : Firing 1<\n>
Isr : D = 73<\n>
isr: : Rx 1<\n>
CONTROL_limitSpeed_run : Front: 73(0) Rear: 43(1)<\n>
CONTROL_limitSpeed_run : Front: 73(1) Rear: 43(1)<\n>
US_driverIn: : Firing 2<\n>
```



But we still see the
flip of sensors...

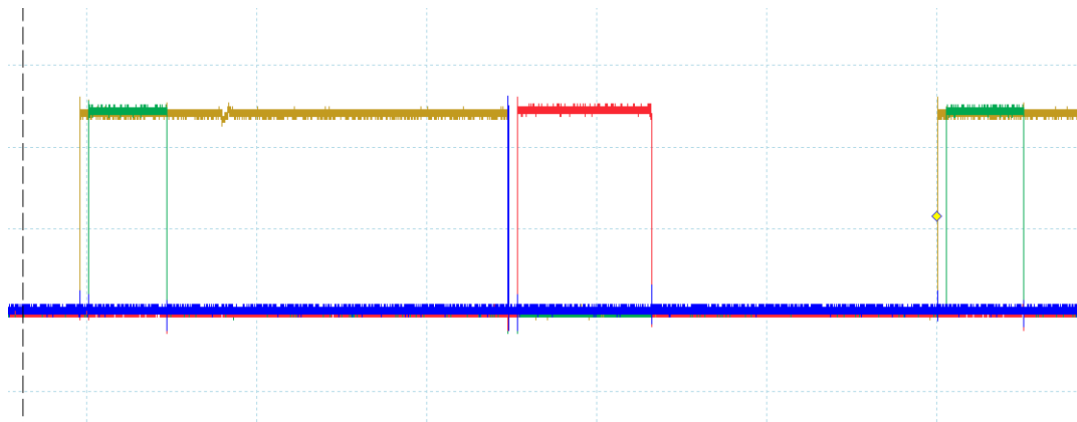
Let's add some hardware debug tools



Physically, this is the rear signal (2)!

After 4 hours of debugging...

- Making the signals volatile increases stability of the status (minor bug)
- Adding some error handlers increased the reliability of the signals but did not solve the root cause problem
- Adding Pin toggles to the drivers to see if the correct Ultrasonic sensor is fired – it is
- Remaining part – The Timer peripheral....
- But also a hardcore init inside the ISR did not solve the problem

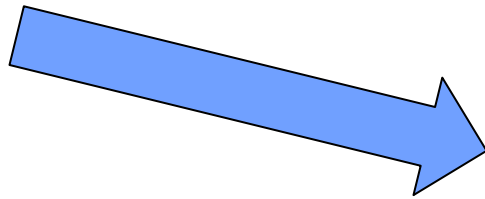
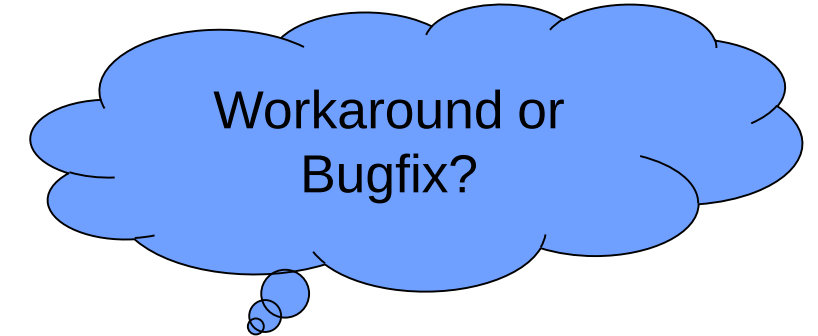
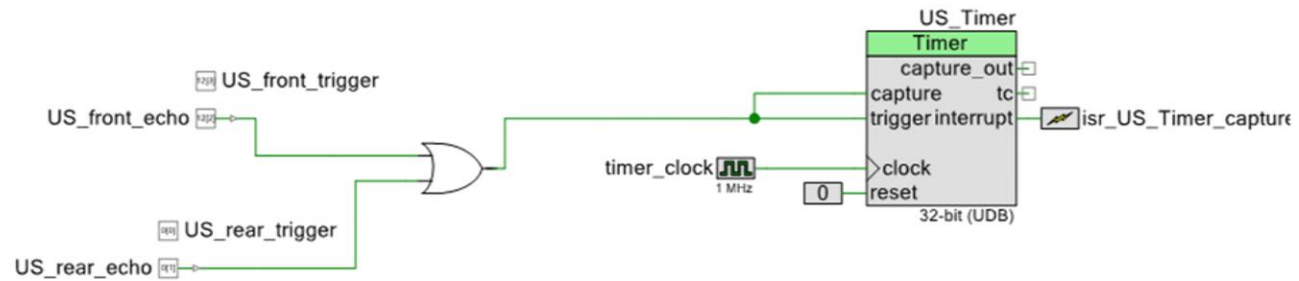


Let's blame the hardware....

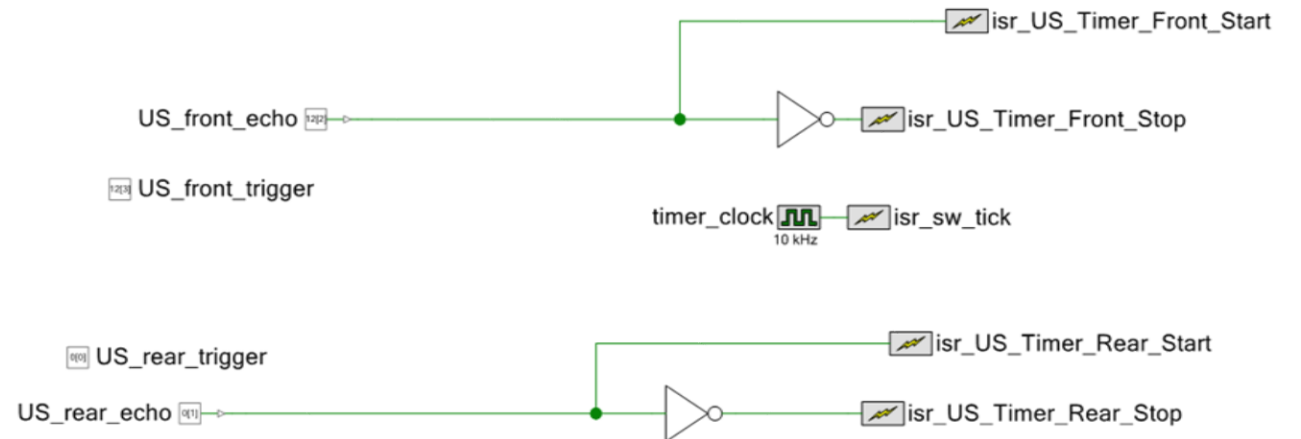
What can we do?

- Workaround would be to use the synchronous driver (accepting the real time penalty)
- Or provide individual timers per sensor
 - This would be the best solution
 - Unfortunately we do not have sufficient UDB blocks for this
- However, setting up a test system with individual timer revealed another weakness
 - The timer is only reset in case the TC (reached 0) bit was not set.
 - However, we still might have a valid capture.
 - Correcting this design flaw, further increased stability, but did not solve the problem.
- Therefore at the end of the day...
- Used a different peripheral, i.e. Pin Interrupt + Clock, implementing the timer hardware in software

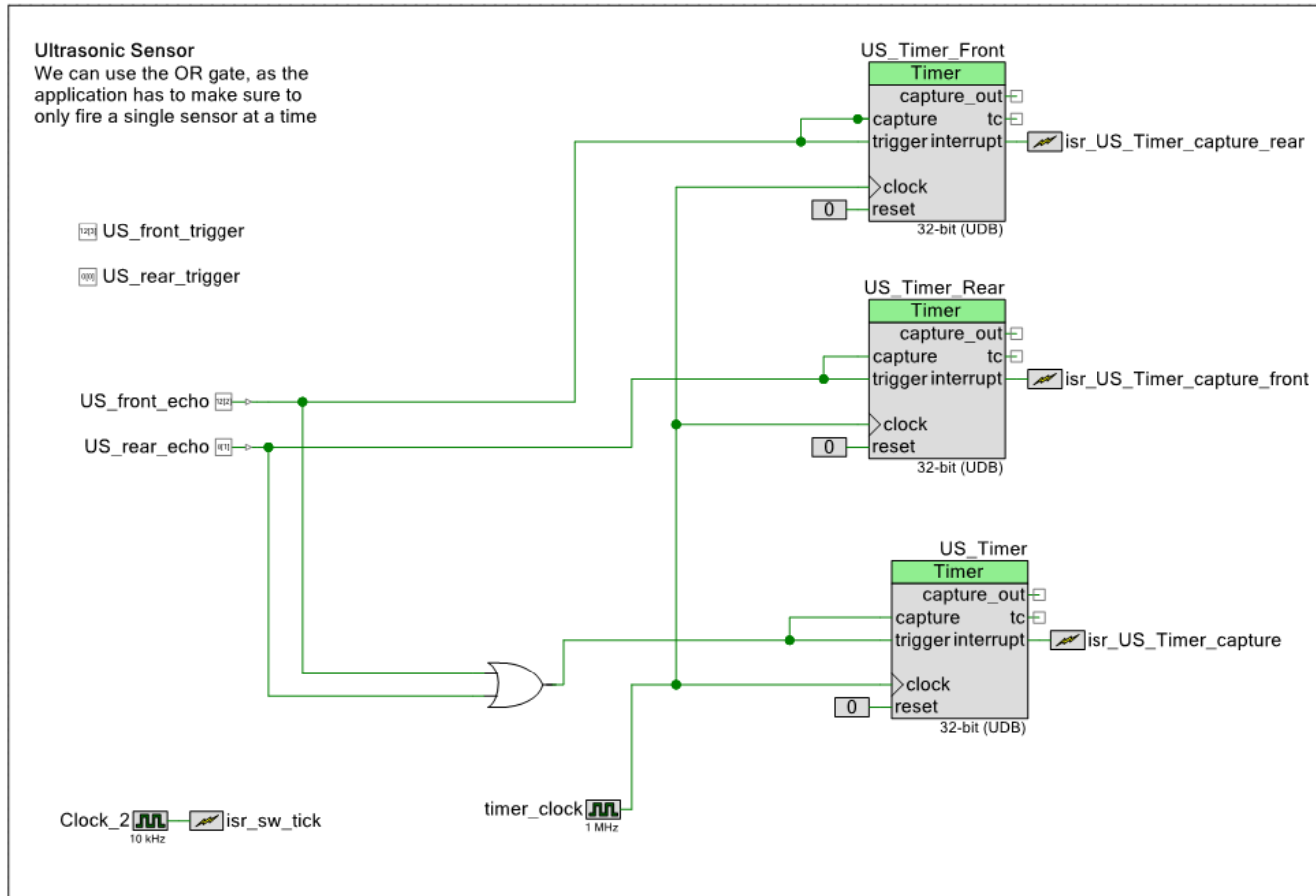
Redesigning the Ultrasonic Hardware Interface



Ultrasonic Sensor



More detailed analysis



We have to create an own debug project!

```
Tech Mapping...  
Error: mpr.M0014: Resource limit: Maximum number of Datapath Cells exceeded (max=24, needed=30). (App=cydsfit)  
Dependency Generation...  
Cleanup...  
Error: fit.M0050: The fitter aborted due to errors, please address all errors and rebuild. (App=cydsfit)  
----- Build Failed: 07/10/2020 09:10:15 -----  
|
```

A first ride



Another Bug Story

CAN

COM Task

Control Task

Decoder 1..4

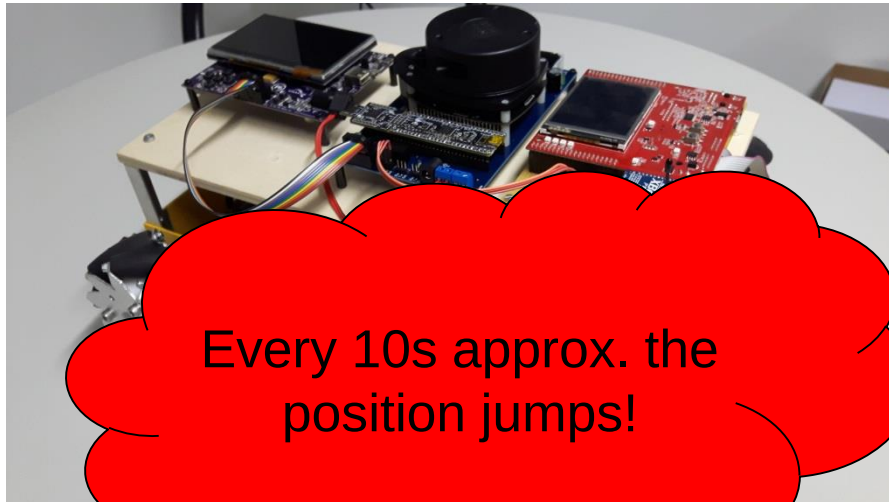
Decoder
Signal

Some magic math

$$\begin{matrix} dx \\ dy \\ dphi \end{matrix} = \begin{bmatrix} 1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 1 & \dots & 0 \end{bmatrix} \begin{matrix} w1 \\ w2 \\ w3 \\ w4 \end{matrix}$$

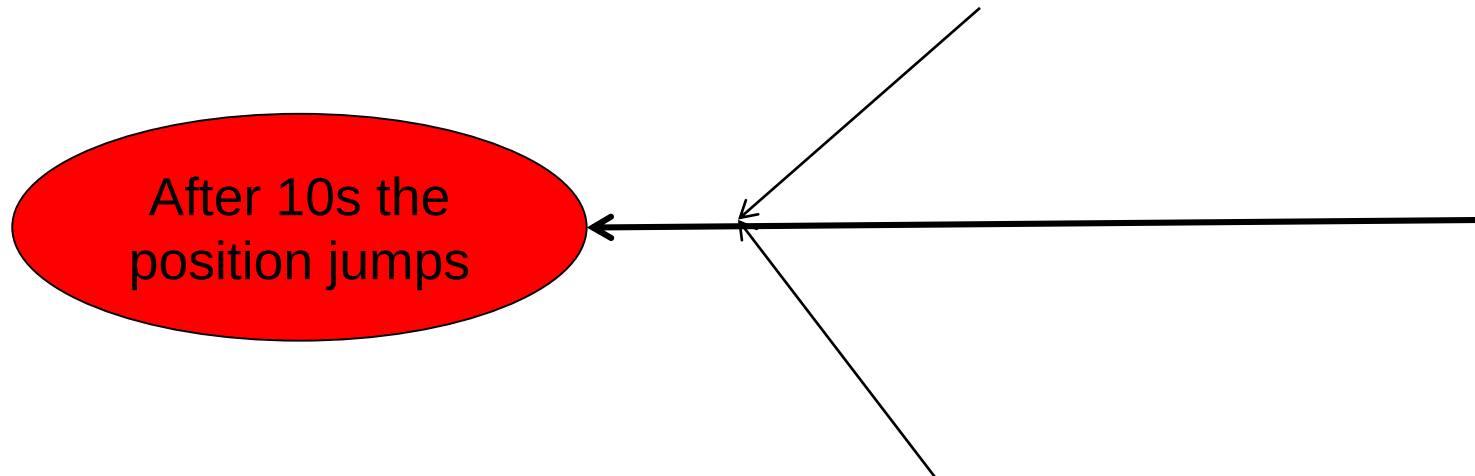
$$\begin{matrix} x \\ y \\ phi \end{matrix} = \int \begin{matrix} dx \\ dy \\ dphi \end{matrix} \sin / \cos dt$$

Position
Signal



Analysis

- What are potential (root)causes?
- What information do we need for the analysis?



Approach

Obviously we have to follow the signal flow

- Decoder → CAN Protocol → CANOPEN Decoder Object → Signal Decoder Object → Signal Position Object
- We can either start with the Decoder (bottom up)
- or the Signal Position (top down)
- or somewhere in the middle (slicing)

As the analysis of the CAN protocol itself is pretty tedious, we start by analysing the signal data.

Analysis of the decoder signal



Workaround 1 – Add filter in the decoder signal

```
//In the first run, set the old values to the current decoder
//value to avoid a wrong delta

if (praw->firstRun == FALSE)
{
    //Calculate the application data as delta displacement
    //as compared to the last position
    //Update previous position of FL with current position
    //Filter out "jumping values"

    if (_abs((praw->engineFL_Pos - praw->engineFL_Pos_prev))
        < ENGINEDECODER_INVALID)
    {
        pvalue->delta_displacementFL = (float32_t)
            (praw->engineFL_Pos - praw->engineFL_Pos_prev)
            * ENGINEDECODER_WHEELDIAMETER * M_PI /
            ENGINEDECODER_COUNTSPERREVOLUTION;
        prawWrite->engineFL_Pos_prev = praw->engineFL_Pos;
    }
    else
    {
        //wrong value, to be ignored
        asm("nop");
    }
}
```

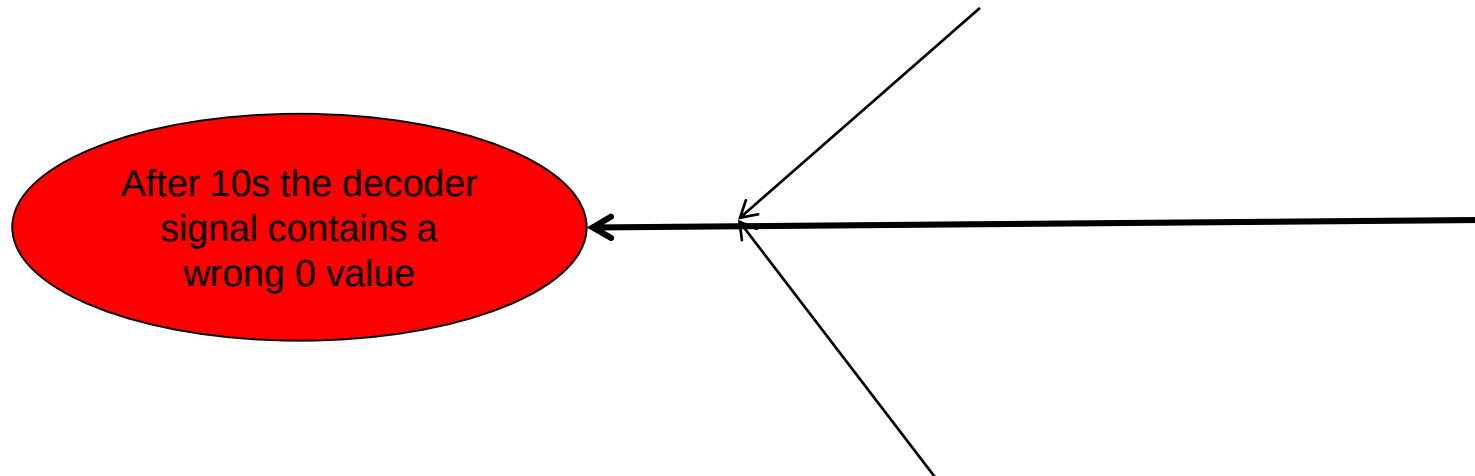
Result



Root Cause Analysis

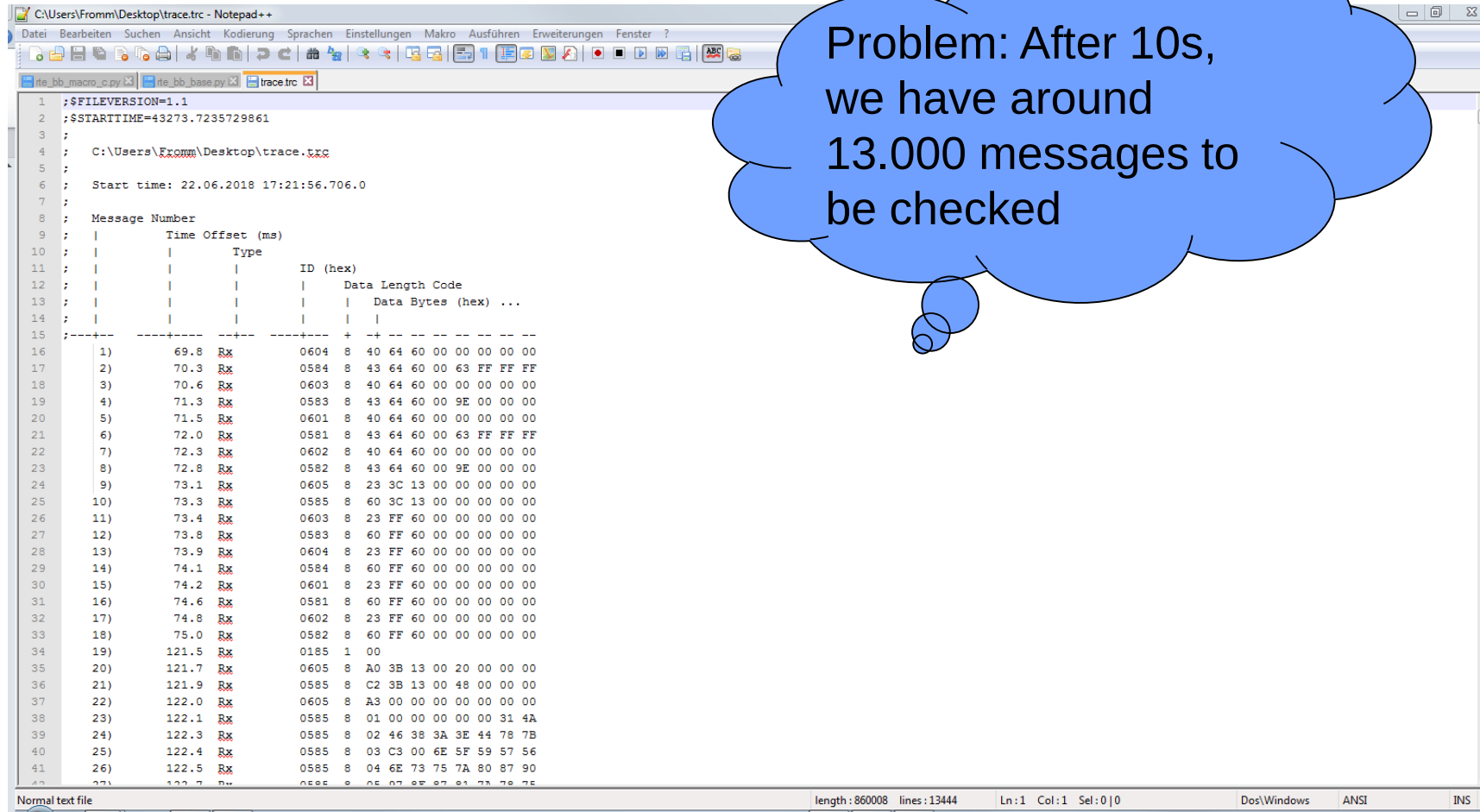
Although the workaround provides a much better signal, it still is a hack

- Every x-seconds, we miss one signal
- What happens, if the spikes are not that prominent any more?



Brute force

Checking the CAN Signal

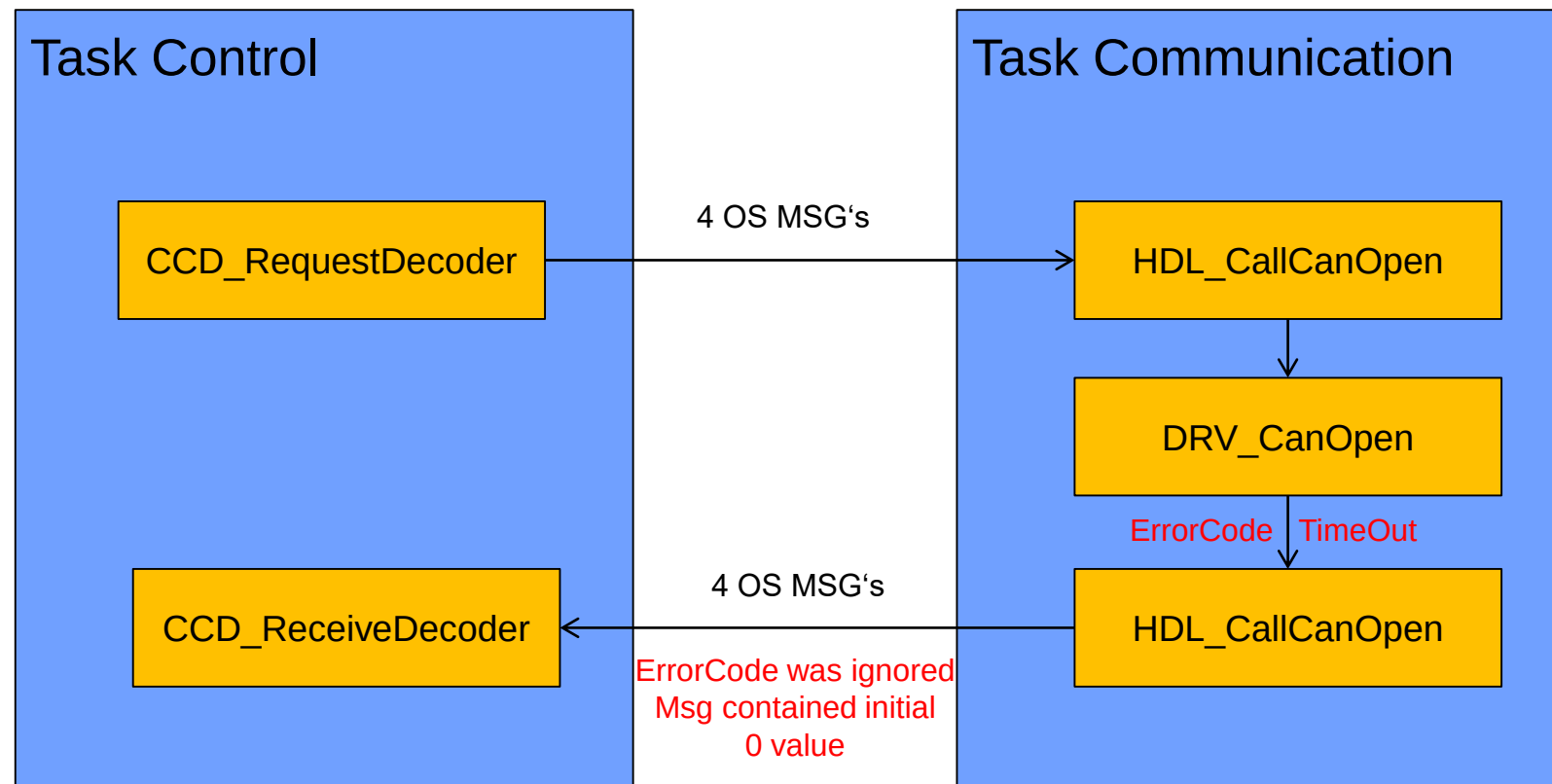


Problem: After 10s,
we have around
13.000 messages to
be checked

```
1 ;$FILEVERSION=1.1
2 ;$STARTTIME=43273.7235729861
3 ;
4 ; C:\Users\Fromm\Desktop\trace.trc
5 ;
6 ; Start time: 22.06.2018 17:21:56.706.0
7 ;
8 ; Message Number
9 ; | Time Offset (ms)
10 ; | Type
11 ; | ID (hex)
12 ; | Data Length Code
13 ; | Data Bytes (hex) ...
14 ;
15 ;-----+-----+-----+-----+-----+-----+
16 1) 69.8 Rx 0604 8 40 64 60 00 00 00 00 00
17 2) 70.3 Rx 0584 8 43 64 60 00 63 FF FF FF
18 3) 70.6 Rx 0603 8 40 64 60 00 00 00 00 00
19 4) 71.3 Rx 0583 8 43 64 60 00 9E 00 00 00
20 5) 71.5 Rx 0601 8 40 64 60 00 00 00 00 00
21 6) 72.0 Rx 0581 8 43 64 60 00 63 FF FF FF
22 7) 72.3 Rx 0602 8 40 64 60 00 00 00 00 00
23 8) 72.8 Rx 0582 8 43 64 60 00 9E 00 00 00
24 9) 73.1 Rx 0605 8 23 3C 13 00 00 00 00 00
25 10) 73.3 Rx 0585 8 60 3C 13 00 00 00 00 00
26 11) 73.4 Rx 0603 8 23 FF 60 00 00 00 00 00
27 12) 73.8 Rx 0583 8 60 FF 60 00 00 00 00 00
28 13) 73.9 Rx 0604 8 23 FF 60 00 00 00 00 00
29 14) 74.1 Rx 0584 8 60 FF 60 00 00 00 00 00
30 15) 74.2 Rx 0601 8 23 FF 60 00 00 00 00 00
31 16) 74.6 Rx 0581 8 60 FF 60 00 00 00 00 00
32 17) 74.8 Rx 0602 8 23 FF 60 00 00 00 00 00
33 18) 75.0 Rx 0582 8 60 FF 60 00 00 00 00 00
34 19) 121.5 Rx 0185 1 00
35 20) 121.7 Rx 0605 8 A0 3B 13 00 20 00 00 00
36 21) 121.9 Rx 0585 8 C2 3B 13 00 48 00 00 00
37 22) 122.0 Rx 0605 8 A3 00 00 00 00 00 00 00
38 23) 122.1 Rx 0585 8 01 00 00 00 00 00 31 4A
39 24) 122.3 Rx 0585 8 02 46 38 3A 3E 44 78 7B
40 25) 122.4 Rx 0585 8 03 C3 00 6E 5F 59 57 56
41 26) 122.5 Rx 0585 8 04 6E 73 75 7A 80 87 90
```

Code Review

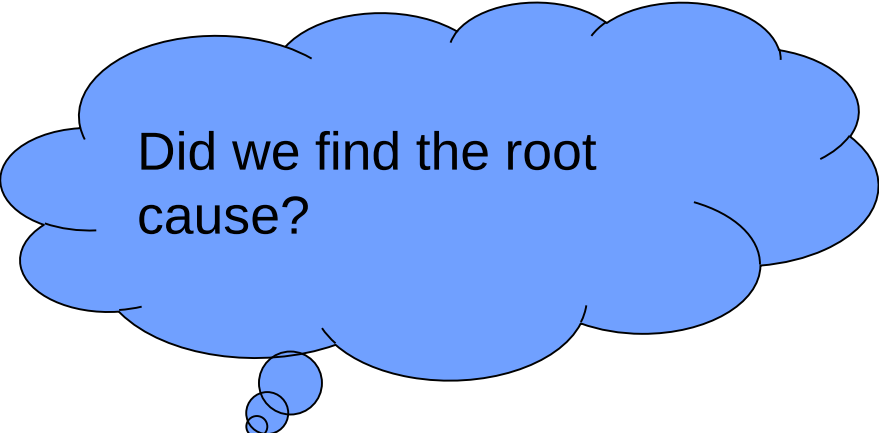
- The complex driver, which received the CanOpen message objects through an OS message call, ignored the driver return code.



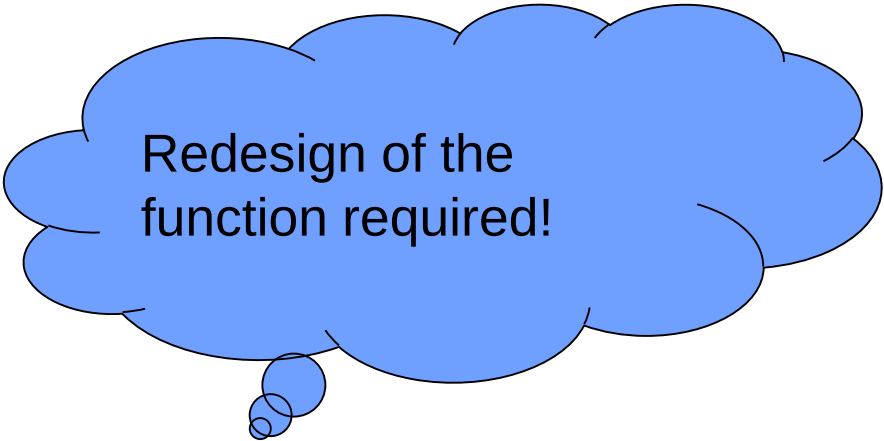
Workaround 2 – Consider error code

```
RC_t MSG_rx_decoderFromCanopen( void * const pInPayload, uint32_t inPayloadSize, void * const
pOutPayload, uint32_t outPayloadSize , uint16_t rxstatus)
{
    RC_t result = RC_SUCCESS;

    if (0 != rxstatus)
    {
        //error happened during driver call
        //In this case do not update the async buffer
        //But wait for the next request message
        return RC_ERROR_BAD_DATA;
    }
}
```



Did we find the root cause?



Redesign of the function required!

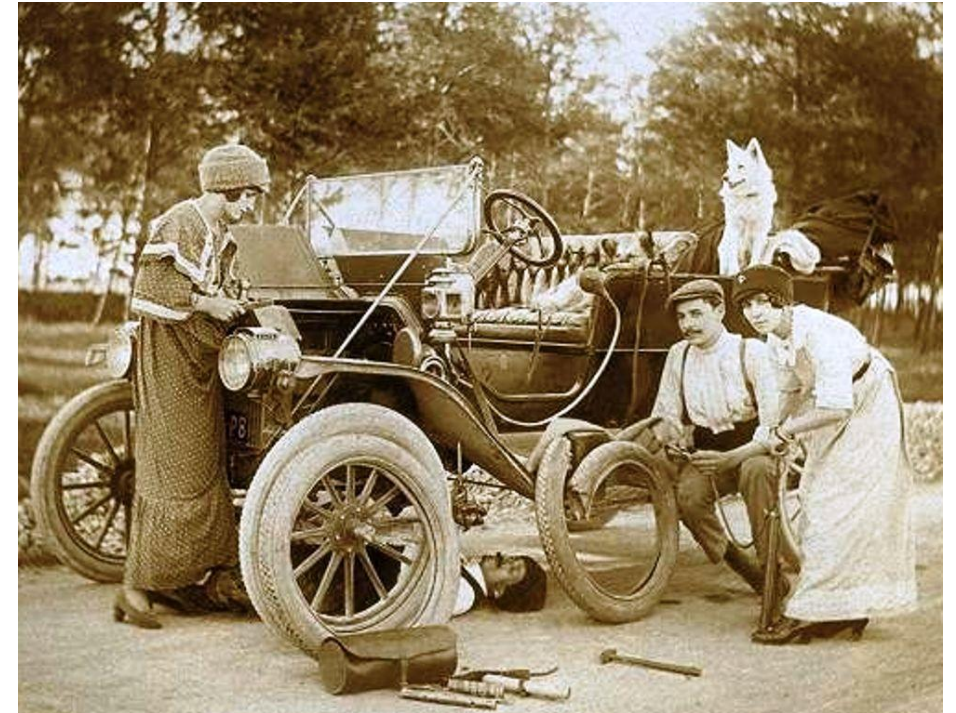
Top 10 rules for efficient debugging

- Work systematically
 - Get the problem reproducible,
 - then localize the place where it goes wrong,
 - then find the root cause
- Use debugging techniques like
 - slicing
 - tracing
 - stubbing
 - hooks
- Know what the program does, do not guess
- Bring in a second pair of eyes early
- Even if the program behaves ok, it may be buggy!
- And finally, after you changed the code, check for side effects

And never ignore return values!!!!

6 Stages of Debugging

- That can't happen
- That doesn't happen in my system
- That shouldn't happen
- Why does that happen?
- Oh, I see
- How did that ever work

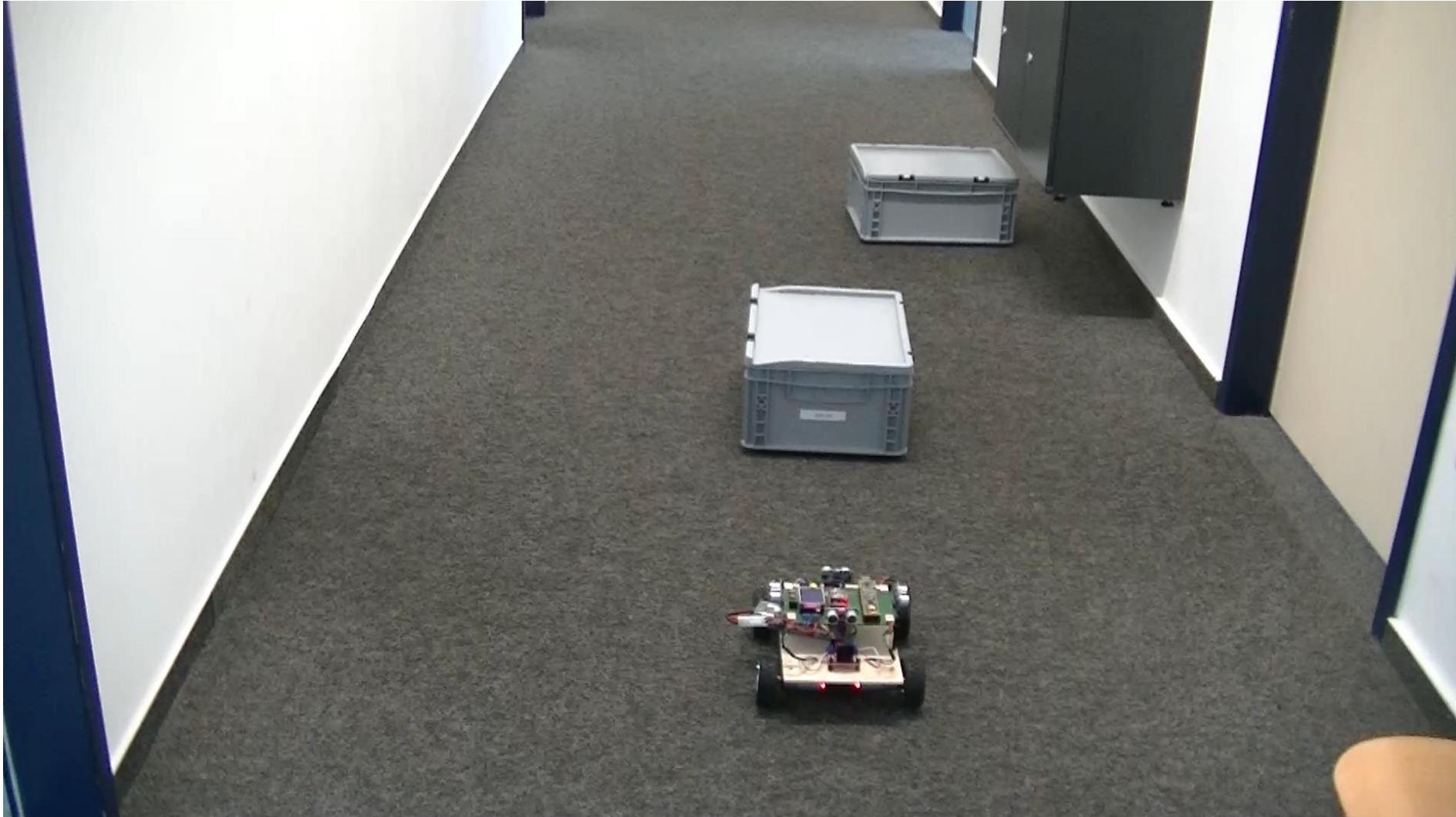


Wrapup

- Many problems are only found in the integrated system
 - Unit testing is required, but not sufficient!
 - Stress-tests and long running tests are mandatory in addition
- Debugging is engineering!
 - Develop a debug strategy
 - Add debug code
- Debugging helps you to understand the system
 - Before the debugging, I thought I knew what the architecture is doing
 - Afterwards, I understood much more (although probably not 100%)
 - Very often, design flaws which are not related to the main problem can be improved during debugging

Some more testrides

Difficult descision!



Sensor stopped working – again!



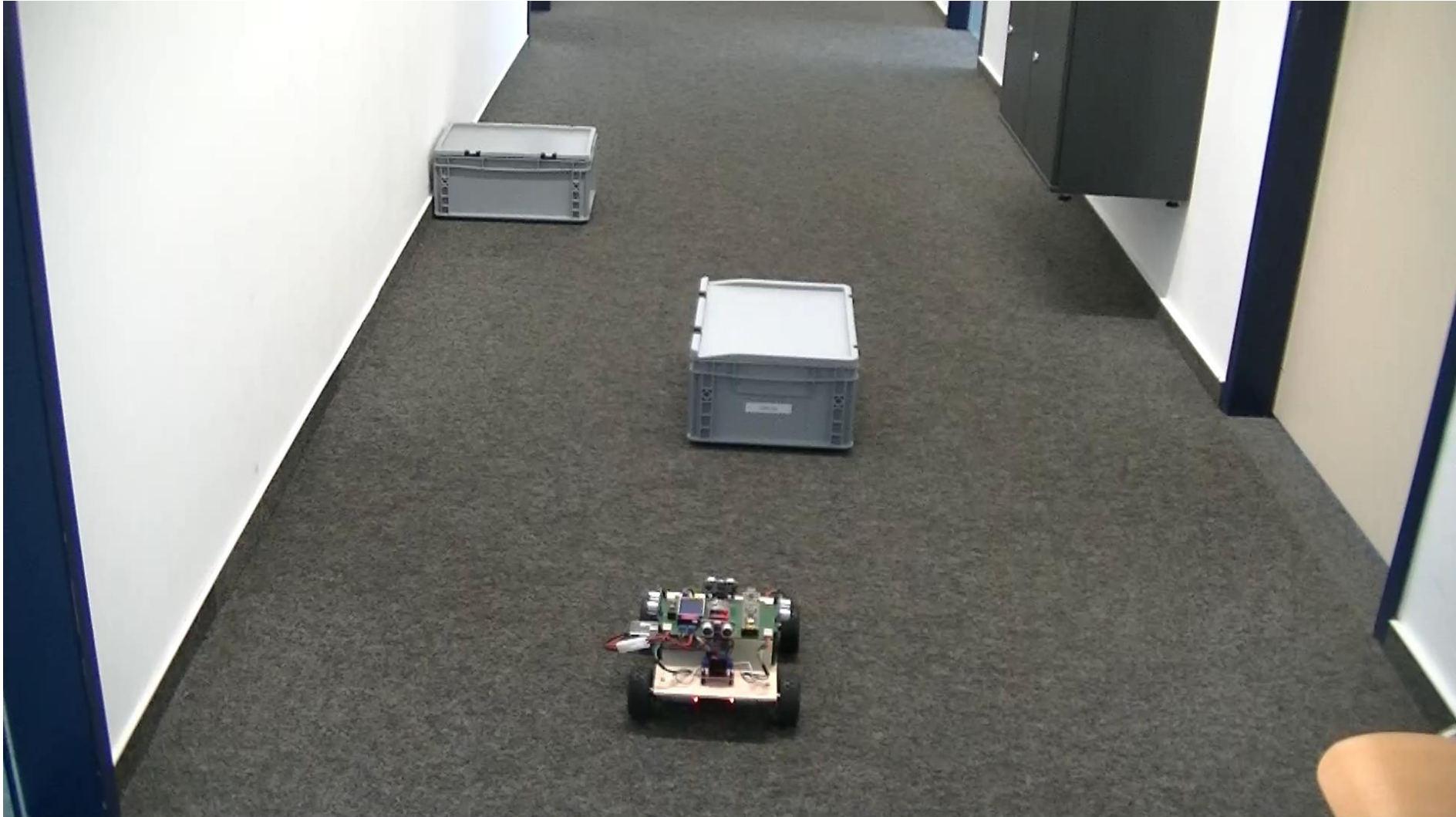
OK, but not intended



At least the engine blocking safety feature is working



I can fly



Still something to do for the next years!

